SOUTH LAKE HOME EDUCATORS



Lesson 2

Programming Basics I: Sequences, Events & the Game Loop

Thursday, August 20, 2026

At a Glance

Lesson 2 objectives & materials

By the end of today, students will:

- Explain sequences and why order matters
- Explain events and give a game example
- Describe the game loop concept
- Build a working Key Press Reactor - the start of the Semester 1 project

What We're Using Today

- Demo laptop with all 8 engines verified
- Scratch, for the live lecture demo
- Engine-specific Student Handouts, one per student
- This lesson's Assets folder (placeholder sprite + sound - no download needed)
- Your own Game Design Document, to start filling in during lab

IGDA framework tie-in: Programming - control structures and event-driven programming (introductory).

Today's Agenda

0:00 - 0:05	Welcome back, Lesson 1 recap, quick install spot-check
0:05 - 0:20	Sequences (live demo in Scratch)
0:20 - 0:35	Events (live demo in Scratch)
0:35 - 0:50	The Game Loop (concept + "under the hood" look)
0:50 - 1:00	Introduce lab exercise, hand out engine-specific handouts
LAB TIME	***— Key Press Reactor —***
1:00 - 1:50	Build the Key Press Reactor - the start of your Semester 1 project
1:50 - 2:00	Exit check: quick share-out



Welcome Back

Lesson 1 recap and a quick install spot-check before we dive in.

5 min

Welcome Back

5 min

Quick recap of Lesson 1:

- Internet safety, what game development involves, and a tour of all 8 engines
- Everyone chose (or shortlisted) an engine and started installing it
- Today: our first real programming lesson - and the start of your Semester 1 project

If you didn't finish installing last week, that's okay - we've got a few minutes now, and Scratch and GDevelop work straight in a browser with zero install if you're in a pinch.



Sequences

The simplest idea in programming: instructions that happen one after another, in order.

15 min

Sequences

15 min

A sequence is a list of instructions that happen one after another, in order.

Think about your morning routine - wake up, get dressed, eat breakfast. If you get dressed before you wake up, that doesn't work. Order matters.

Live demo (Scratch):

```
move 10 steps  →  turn 15 degrees  →  say "Hello!"
```

Stacked blocks run top to bottom, one after another - every language we use this year (GDScript, C#, GML...) runs sequences like this at its core.



Discussion: what's something else in your daily life that has to happen in a specific order to work?



Events

Code that says "when this happens, do that" - the shift from running once to reacting.

15 min

Events

15 min

An event is code that waits, then reacts: "when this happens, do that."

Live demo (Scratch):

```
when [right arrow] key pressed  →  move 10 steps
```

Nothing happens until the key is pressed - and each press runs the whole sequence underneath it again.

Same idea, different names, across every engine:

Scratch	"when [key] pressed" hat block
Godot	signals, or checking Input inside _process()
Unity	Keyboard.current checks inside Update()
GameMaker	the Key Pressed / Key Down event



What other events can you think of in games you've played?

Sequences vs. Events - side by side

Sequences vs. Events

SEQUENCE

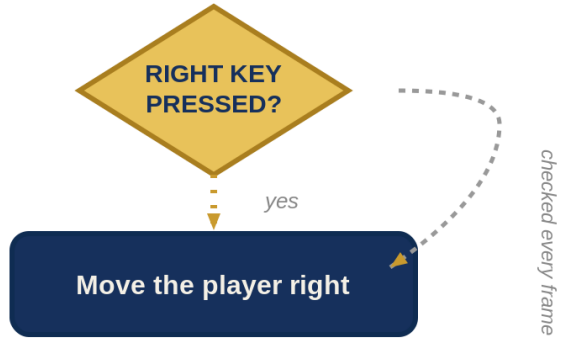
runs once, top to bottom, every time



Predictable, fixed order.
Nothing skips ahead or waits.

EVENT

waits, then reacts - maybe never, maybe many times



Only runs when its trigger fires.
The rest of the time, it just waits.

Your Key Press Reactor this week is one Event. Setting up the character before that is a Sequence.



The Game Loop

How does a character keep moving smoothly while you hold a key? This is why.

15 min

The Game Loop

15 min

The engine isn't waiting for one event and stopping - it's constantly asking, ~30 to 60 times every second:

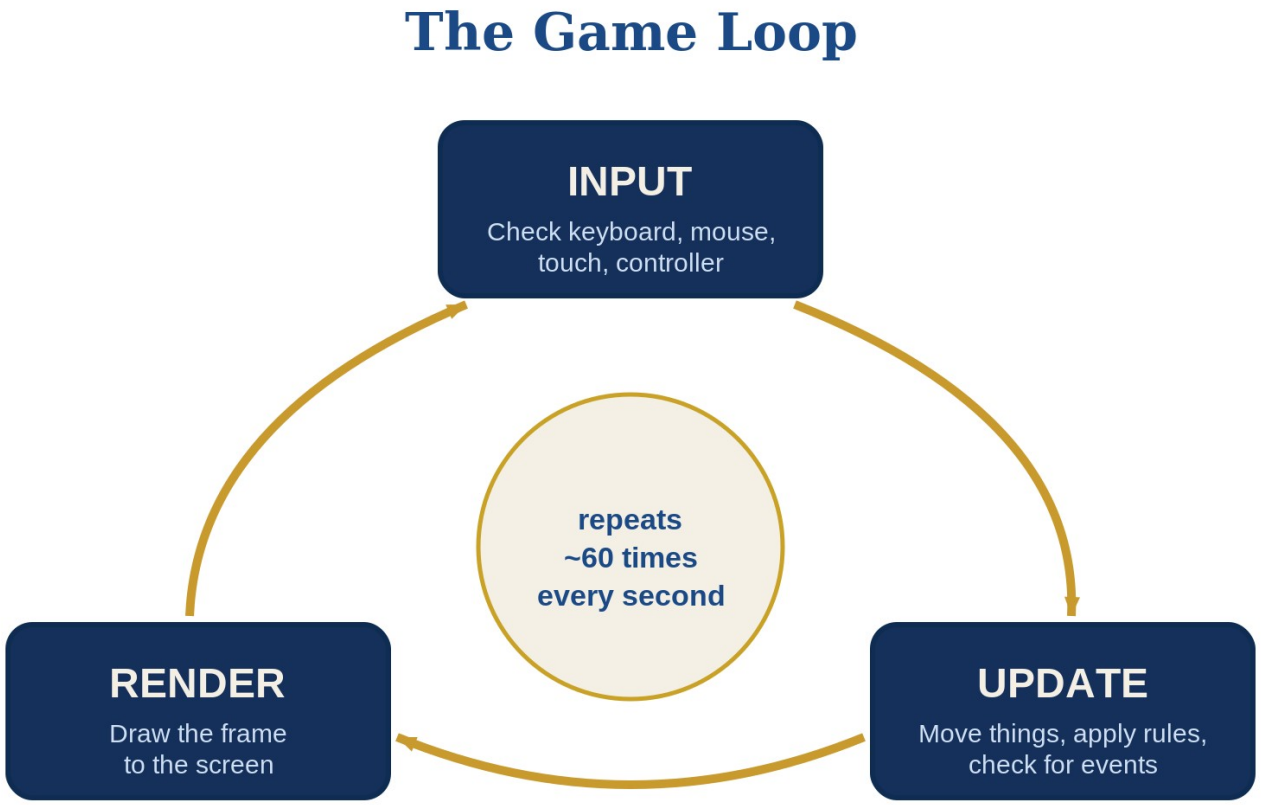
What's the current input?	**What needs to move?**	**What needs to be redrawn?**

That loop - check, update, draw, check, update, draw - happening dozens of times a second, is the game loop. You don't usually write it yourself; the engine runs it for you. Almost everything you build this year runs inside it.



Flip-book analogy: each page is slightly different, and flipping through them fast makes it look like smooth motion. Each "page" is one pass through the game loop.

The Game Loop - Input, Update, Render



Every engine you use runs some version of this loop under the hood - it's why your character moves smoothly instead of jumping in one big step.

Bonus: Diagrams Programmers Use

BONUS

You just described the Game Loop in words. Professional game and software developers often draw it instead - using a couple of simple, standard diagram styles from UML (Unified Modeling Language).



Activity Diagram

Shows the order things happen in - like a flowchart. Next up: your Key Press Reactor as one.



State Diagram

Shows the different "modes" something can be in, and what switches it between them - like Idle vs. Moving.

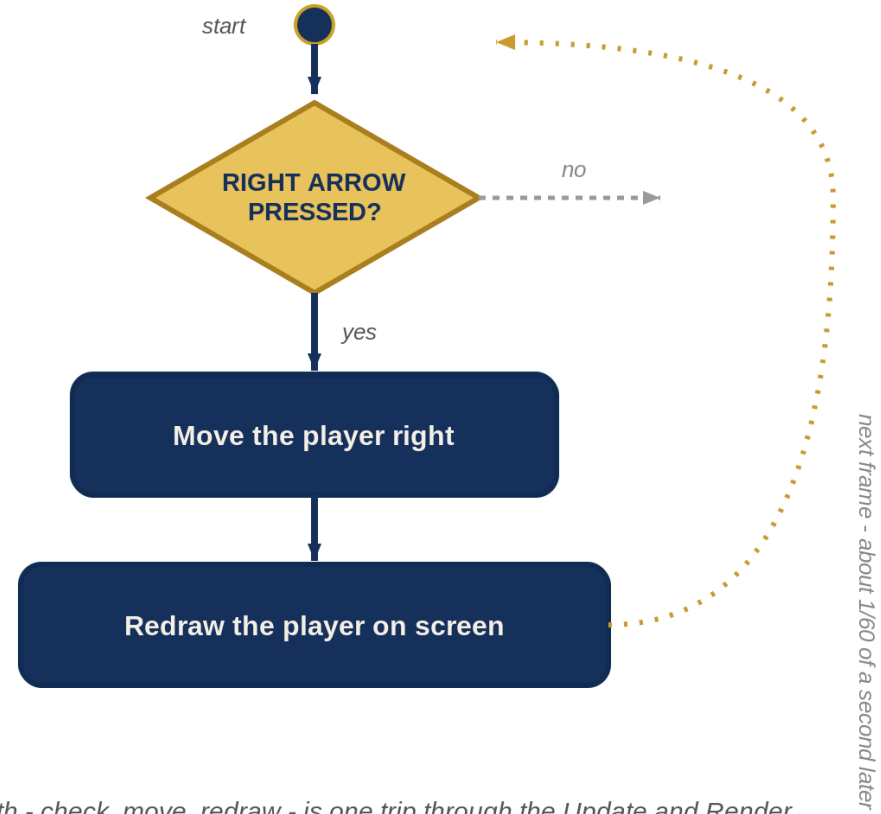
We'll come back to a third kind - Class Diagrams - later in the year, once you're writing more object-oriented code.

Activity Diagram - Key Press Reactor

BONUS

Activity Diagram: Key Press Reactor

a simplified UML-style diagram of what happens, in order, each time the game checks your input



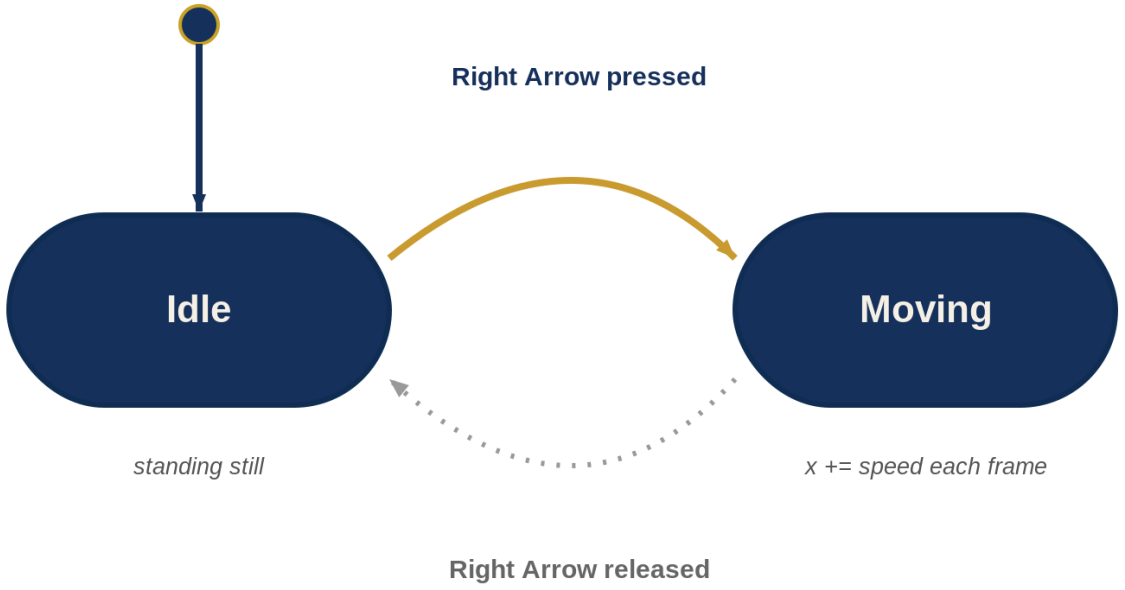
This whole path - check, move, redraw - is one trip through the Update and Render steps of the Game Loop you already learned. An Activity Diagram is just a picture of that order.

State Diagram - The Player Character

BONUS

State Diagram: The Player Character

the different "modes" your character can be in, and what switches it between them



A game character can usually only be in one state at a time. Level 2 of this week's exercise really has 5 states like this - Idle plus one Moving state per direction.



Today's Exercise & Lab

The Key Press Reactor - and the very first piece of your Semester 1 project.

10 min

The Key Press Reactor

10 min

An object that responds when you press a key - and the very first piece of your Semester 1 project. Save it somewhere you'll remember, as [YourName]Game.

Level 1 - Minimal

One key does one thing.

MVP - a complete finish for today.

Level 2 - Complete

All 4 arrow keys move your character.

Everything works end to end.

Level 3 - Refined

Animation, sound, and/or polish - pick what interests you.

Ready to share.



New this lesson: 4 engines (GameMaker, Unity, Unreal, 001 Game Creator) get a second, alternate set of instructions on their handout - the other coding style (visual or scripted) for the exact same result. Try it if you're curious how the other side works.

Lab Time Notes

- Circulate by engine cluster, starting with whichever engine has the most students this week.
- Common early snag, every engine: an event block/node that isn't actually connected won't run. "Nothing happens" → check connections first.
- Students who finish Level 2 early should be pointed at the Level 3 ideas on their handout - not given new unrelated work.
- At 1:50, call time and do a 2-minute share-out: ask 2-3 students to show what they built, across different engines if possible.



Great Work Today!

Next week: more programming foundations - variables, loops, and conditionals.

Keep building on your Key Press Reactor at home if you'd like - and start jotting notes in your Game Design Document.